

Is the Code Good for Performance?

Short answer: fine for one person's everyday task list, wasteful in specific, fixable spots that only start to hurt once the task list grows or calendar sync is switched on. Every finding below is traced to the exact code doing the extra work — no synthetic benchmarks were run.

App to_do_app (Flutter) Method source read + grep-verified counts, no profiler run

Verdict

At the scale one person's task list actually reaches — tens to a couple hundred tasks, sync off — nothing here will be felt. Hive is fast enough, in-memory sort/filter/group is fast enough, and the UI is simple enough that none of the waste below crosses a visible threshold.

The waste is real, though, and it's **structural rather than incidental**: several code paths redo full-dataset work in response to a single-item change, and one Google Calendar sync path costs a network round trip per task instead of per sync. Those scale badly — with task-list size, with sync frequency, or with how long the app has been used — in ways a developer testing with a handful of demo tasks would never notice.

3

high-impact findings

3

medium-impact findings

1

low-impact / edge case

Findings

Ordered by impact, not by how easy each is to fix.

Every save rewrites every Hive box, in full

HIGH

`ToDoDataBase.updateDataBase()` is the one save path most task mutations go through, and it unconditionally calls all nine save methods — tasks, all three calendar boxes, categories, hidden categories, settings, sync config, and view-only-calendar config — regardless of which single thing actually changed. Each of the four typed-box saves (`_replaceTaskBox` / `_replaceCalBox`) does

`box.clear()` then `box.addAll(rows)` — a full rewrite of every record, not an update of the one that changed.

VERIFIED

`db.updateDataBase()` is called directly 7 times in `task_page.dart` alone — checkbox toggle, add, delete, undo-restore, edit, category rename, drag-reorder. Toggling one checkbox in a 500-task list re-serializes and re-writes all 500 task records to disk, not 1.

DIRECTION Add per-record save paths (`box.put(key, task)`) for the common single-item mutations, and reserve the full-rewrite path for bulk operations like calendar re-sync.

`data/database.dart`

`pages/task_page.dart`

One sync-progress tick re-sorts, re-filters, and re-groups the entire task list — for every tab

HIGH

`TaskPage.build()` calls `context.watch()` on four providers, including `CalendarSyncProvider`. Any of their `notifyListeners()` calls rebuilds the whole page. Because `TabBarView` takes an eager `children` list — `_taskCategoryTabs().map((tab) => _buildTasksForTab(...)).toList()` — **every** tab (All, each category, High/Medium/Low) runs its full pipeline on every rebuild, not just the one currently visible: sort the whole list, filter it, search it, sort it again, then group it.

VERIFIED

`CalendarSyncProvider.updateProgress()` is called at 0.3 / 0.5 / 0.8 during one sync run, plus `startSync()` and `finishSync()` — 5 notifies per sync. With, say, 8 tabs (All + 4 categories + 3 priorities), one calendar sync run triggers roughly **40 full sort → filter → group passes** over the complete task list.

DIRECTION Scope the sync-progress watch to just the sync-bar widget instead of the whole page, and cache each tab's computed list, invalidating only when the task list or that tab's inputs actually change.

`pages/task_page.dart`

`providers/calendar_sync_provider.dart`

Saving a synced task re-fetches the whole calendar first

HIGH

`GoogleCalendarService.addOrUpdateEvent()` calls `getEvents(calendarId)` — a live Calendar API request across a roughly 120-day window — every time it runs, then linearly scans the result with `firstWhere` to find a matching event by id. This happens on every single task add or edit while Google sync is enabled, even though the task already carries its own Google event id in `remoteEventIds[1]` once it's been synced once.

VERIFIED

`services/google_calendar_service.dart` — the network fetch is unconditional inside `addOrUpdateEvent`, not cached or short-circuited by an existing id. Editing 10 tasks in a row with sync on means 10 separate full-window calendar fetches, not 1.

DIRECTION Once a task has a stored event id, call the Calendar API's direct update-by-id instead of list-then-match; reserve the list fetch for first-time linking.

services/google_calendar_service.dart

Every task tile owns an audio player that's never released

MEDIUM

`TaskTile` 's state creates `final AudioPlayer _audioPlayer = AudioPlayer();` as a field. There's no `dispose()` override anywhere in the file to release it. Each visible tile holds its own native audio-player resource for its whole lifetime, and none of them are ever cleaned up when the tile is removed from the tree.

VERIFIED

Grep for `dispose` in `components/task_tile.dart` returns nothing — the `State` class doesn't override it at all.

DIRECTION Either add a `dispose()` that disposes `_audioPlayer`, or share one `AudioPlayer` instance across tiles instead of one per tile.

components/task_tile.dart

Date/month/year groups are built eagerly, not lazily

MEDIUM

The outer list in `_buildTasksForTab` is a plain `ListView(children: ...)`, not `ListView.builder`. Every group — each with its own `Card`, `ExpansionTile`, and nested `ReorderableListView.builder` for its tasks — is constructed immediately regardless of scroll position. The inner per-task list inside each group is correctly lazy; it's the outer grouping level that isn't.

VERIFIED

`pages/task_page.dart`, the group-rendering `ListView` inside `_buildTasksForTab`.

DIRECTION Switch the outer list to `ListView.builder`, keyed by group. Matters most for "by month"/"by year" grouping over a long task history.

pages/task_page.dart

228 unconditional `print()` calls in services, none gated

MEDIUM

None of the logging in `lib/services/` is wrapped in a debug-only check, so it all runs in release builds too. Individually cheap, but several sit inside per-event import loops during calendar sync, so the cost scales with how much data is moving through that loop, not with how much a developer actually

needs to see.

VERIFIED

228 `print()` occurrences across 11 files in `lib/services/`; 0 `kDebugMode` occurrences in the same directory. Heaviest: `local_calendar_service.dart` (47), `google_calendar_service.dart` (46), `outlook_calendar_service.dart` (44), `notification_service.dart` (39).

DIRECTION Route through a small logger that's a no-op in release builds, or gate the loop-internal ones behind `kDebugMode` the way `main.dart` already does.

`services/*.dart`

Missed-repeat backfill scans the task list once per missed occurrence

LOW / EDGE CASE

`RepeatTask.createPendingRepeatTasks` loops (capped at 1000 iterations) once per missed occurrence of a repeating task, and each iteration does an `indexWhere` scan over the whole task list plus a notification-scheduling call. A daily task neglected for a year means up to 365 iterations, each an $O(\text{task-count})$ scan.

VERIFIED

`services/repeat_task.dart`. Rare in practice — most repeat gaps are small — but unbounded relative to list size.

DIRECTION Low priority; worth a bound on total work per catch-up pass if it's ever reported as slow.

`services/repeat_task.dart`

What's actually fine

Worth naming so the findings above don't read as "nothing here is sound."

Hive is the right call for this app's realistic local data volume — the seven findings above are about the *access pattern* around it, not the database choice itself.

The per-task list inside each group already uses `ReorderableListView.builder` correctly — genuinely lazy, builds only what's visible.

Search is a plain substring scan on name + note — appropriately simple for this data scale; nothing fancier is needed here.

No heavy image/asset work on any hot path — the Lottie confetti only plays on a discrete completion

event, not continuously, and calendar-provider icons are small static PNGs.

flutter analyze is clean — no lurking type errors forcing defensive runtime checks or silent `dynamic` fallbacks in a hot path.